

Manual paso a paso Web_Sesion13

**CRUD con modal y AJAX · consultas de valor con gráfico ·
paginación**

Construcción, archivo por archivo, de un sitio web sobre la base de datos VentasLeon. Continúa el proyecto de la Sesión 12 y lo amplía con las tablas de Proveedor y Ubigeo, un CRUD completo con combos en cascada, un gráfico estadístico con Chart.js y la paginación local y con procedimiento almacenado. Diseñado para ir copiando y pegando cada paso.

Contenido

1. Punto de partida: de Web_Sesion12 a Web_Sesion13
2. Ampliar la base de datos (VentasLeon_S13.sql)
3. Nuevos modelos: Proveedor y Ubigeo
4. Actualizar el DbContext
5. Ampliar la capa de datos (repositorio)
6. Diseño: navbar, estilos y página de inicio
7. Ejemplo 1 · CRUD de Proveedores (tabla, modal y AJAX)
8. Ejemplo 2 · Consulta de valor con gráfico (Chart.js)
9. Ejemplo 3 · Paginación (local y con procedimiento)
10. Probar la aplicación

Cómo usar este manual: sigue las páginas en orden. Cada bloque oscuro es código para copiar y pegar tal cual en el archivo indicado encima (la ruta naranja). Este manual continúa el proyecto de la Sesión 12.

1. Punto de partida: de Web_Sesion12 a Web_Sesion13

La Sesión 13 continúa el sitio de la Sesión 12. Partimos de una copia del proyecto y solo le agregamos lo nuevo: las tablas de Proveedor y Ubigeo, y tres ejemplos.

Paso 1 Copie la carpeta del proyecto `Web_Sesion12` y renómbrela a `Web_Sesion13` (carpeta, archivo `.csproj` y la solución).

Paso 2 Abra el proyecto y reemplace el **namespace** `Web_Sesion12` por `Web_Sesion13` en todo el proyecto (Editar → Buscar y reemplazar en toda la solución).

Paso 3 Verifique que los paquetes NuGet sigan instalados:

NuGet

```
Microsoft.EntityFrameworkCore.SqlServer  
Microsoft.EntityFrameworkCore.Tools
```

Paso 4 El archivo de proyecto debe quedar así:

Web_Sesion13 \ Web_Sesion13.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">  
  
  <PropertyGroup>  
    <TargetFramework>net10.0</TargetFramework>  
    <Nullable>enable</Nullable>  
    <ImplicitUsings>enable</ImplicitUsings>  
    <RootNamespace>Web_Sesion13</RootNamespace>  
  </PropertyGroup>  
  
  <ItemGroup>  
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="10.0.0" />  
    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="10.0.0">  
      <PrivateAssets>all</PrivateAssets>  
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>  
    </PackageReference>  
  </ItemGroup>  
  
</Project>
```

Si prefiere empezar de cero, cree un proyecto **ASP.NET Core Web App (Razor Pages)** .NET 10 llamado **Web_Sesion13**, instale los dos paquetes y siga el manual de la Sesión 12 para la base, luego continúe aquí.

2. Ampliar la base de datos (VentasLeon_S13.sql)

Sobre la base **VentasLeon** (ya creada en la Sesión 12) ejecute **una sola vez** el script `VentasLeon_S13.sql`. Agrega las tablas de Ubigeo y Proveedor con datos, más facturas de varios años, la vista y el procedimiento de paginación.

Formato de fecha: el script usa `DATEFROMPARTS` y literales `AAAAAMDD` para que SQL Server los interprete igual aunque esté en español.

SQL \ VentasLeon_S13.sql

```
/* =====
VentasLeon_S13 - Desarrollo de Aplicaciones Basico (Sesion 13)
Ejecute este script UNA sola vez, DESPUES de VentasLeon.sql.
Agrega:
- Ubigeo: Tb_Departamento, Tb_Provincia, Tb_Distrito (con datos)
- Tb_Proveedor (con datos; incluye inactivos para verlos en rojo)
- Mas facturas de ejemplo (varios años) para el grafico y la paginacion
- Vista vw_VistaFacturas y SP usp_ListarFacturas_Paginacion (OFFSET/FETCH)
Fechas: se usan partes enteras (DATEFROMPARTS) para que no dependan del idioma.
===== */

USE VentasLeon;
GO

/* ---- Limpieza por si se vuelve a ejecutar ---- */
IF OBJECT_ID('usp_ListarFacturas_Paginacion') IS NOT NULL DROP PROCEDURE usp_ListarFacturas_Paginacion;
IF OBJECT_ID('vw_VistaFacturas') IS NOT NULL DROP VIEW vw_VistaFacturas;
IF OBJECT_ID('Tb_Proveedor') IS NOT NULL DROP TABLE Tb_Proveedor;
IF OBJECT_ID('Tb_Distrito') IS NOT NULL DROP TABLE Tb_Distrito;
IF OBJECT_ID('Tb_Provincia') IS NOT NULL DROP TABLE Tb_Provincia;
IF OBJECT_ID('Tb_Departamento') IS NOT NULL DROP TABLE Tb_Departamento;
GO

/* ===== UBIGEO ===== */
CREATE TABLE Tb_Departamento (
    Cod_dep    VARCHAR(2)    NOT NULL PRIMARY KEY,
    Nom_dep    VARCHAR(60)   NOT NULL
);

CREATE TABLE Tb_Provincia (
    Cod_prov   VARCHAR(4)    NOT NULL PRIMARY KEY,
    Cod_dep    VARCHAR(2)    NOT NULL REFERENCES Tb_Departamento(Cod_dep),
    Nom_prov   VARCHAR(60)   NOT NULL
);

CREATE TABLE Tb_Distrito (
    Cod_dist   VARCHAR(6)    NOT NULL PRIMARY KEY,
    Cod_prov   VARCHAR(4)    NOT NULL REFERENCES Tb_Provincia(Cod_prov),
    Nom_dist   VARCHAR(60)   NOT NULL
);
GO

INSERT INTO Tb_Departamento (Cod_dep, Nom_dep) VALUES
('15','Lima'),
('08','Cusco'),
('04','Arequipa');

INSERT INTO Tb_Provincia (Cod_prov, Cod_dep, Nom_prov) VALUES
('1501','15','Lima'),
('1507','15','Huaral'),
('0801','08','Cusco'),
('0808','08','Urubamba'),
('0401','04','Arequipa'),
('0405','04','Camana');

INSERT INTO Tb_Distrito (Cod_dist, Cod_prov, Nom_dist) VALUES
('150101','1501','Lima (Cercado)'),
('150122','1501','Miraflores'),
('150130','1501','San Isidro'),
('150701','1507','Huaral'),
('150702','1507','Aucallama'),
('080101','0801','Cusco'),
```

```

('080107','0801','Wanchaq'),
('080801','0808','Urubamba'),
('080805','0808','Ollantaytambo'),
('040101','0401','Arequipa'),
('040129','0401','Yanahuara'),
('040501','0405','Camana'),
('040502','0405','Jose Maria Quimper');
GO

/* ===== PROVEEDOR ===== */
CREATE TABLE Tb_Proveedor (
    Cod_prv    VARCHAR(5)    NOT NULL PRIMARY KEY,
    Raz_soc_prv VARCHAR(120) NOT NULL,
    Ruc_prv    VARCHAR(11)   NULL,
    Dir_prv    VARCHAR(150)  NULL,
    Cod_dep    VARCHAR(2)    NULL REFERENCES Tb_Departamento(Cod_dep),
    Cod_prov   VARCHAR(4)    NULL REFERENCES Tb_Provincia(Cod_prov),
    Cod_dist   VARCHAR(6)    NULL REFERENCES Tb_Distrito(Cod_dist),
    Est_prv    INT           NOT NULL DEFAULT 1 -- 1 activo, 0 inactivo
);
GO

INSERT INTO Tb_Proveedor (Cod_prv, Raz_soc_prv, Ruc_prv, Dir_prv, Cod_dep, Cod_prov, Cod_dist, Est_prv) VALUES
('P001','Importaciones Andinas SAC', '20481234567','Av. Colonial 100', '15','1501','150122', 1),
('P002','Suministros del Cusco EIRL', '20512345678','Calle Sol 200', '08','0801','080101', 1),
('P003','Comercial Arequipa SRL', '20456123789','Av. Ejercito 300', '04','0401','040129', 0), -- inactivo
('P004','Distribuidora Lima Norte SAC','20487654321','Av. Tupac Amaru 400', '15','1507','150701', 1),
('P005','Textiles del Sur SA', '20533221100','Jr. Comercio 500', '08','0808','080805', 0), -- inactivo
('P006','Ferreteria Central EIRL', '20441122334','Av. Grau 600', '04','0405','040501', 1),
('P007','Servicios Generales Peru SAC','20499887766','Av. Brasil 700', '15','1501','150130', 1);
GO

/* ===== MAS FACTURAS (varios años) para grafico y paginacion ===== */
/* Genera F006..F040 repartidas en 2023, 2024 y 2025, con su detalle. */
DECLARE @i INT = 6;
WHILE @i <= 40
BEGIN
    DECLARE @num VARCHAR(10) = 'F' + RIGHT('000' + CAST(@i AS VARCHAR(3)), 3);
    DECLARE @anio INT = 2023 + (@i % 3); -- 2023, 2024, 2025
    DECLARE @mes INT = ((@i * 7) % 12) + 1;
    DECLARE @dia INT = ((@i * 3) % 27) + 1;
    DECLARE @fec DATE = DATEFROMPARTS(@anio, @mes, @dia);
    DECLARE @est INT = CASE WHEN @i % 3 = 0 THEN 2 ELSE 1 END; -- cancelada / pendiente
    DECLARE @cli VARCHAR(5) = 'C00' + CAST(((@i % 3) + 1) AS VARCHAR(1));
    DECLARE @ven VARCHAR(5) = 'V00' + CAST(((@i % 2) + 1) AS VARCHAR(1));

    INSERT INTO Tb_Factura (Num_fac, Fec_fac, Fec_can, Est_fac, Cod_cli, Cod_ven, Por_igv)
    VALUES (@num, @fec, CASE WHEN @est = 2 THEN DATEADD(DAY, 10, @fec) ELSE NULL END, @est, @cli, @ven, 18.00);

    INSERT INTO Tb_Detalle_Factura (Num_fac, Cod_pro, Can_ven, Pre_ven) VALUES
        (@num, 'P010', (@i % 3) + 1, 1200.00),
        (@num, 'P020', (@i % 2) + 1, 350.00);

    SET @i = @i + 1;
END
GO

/* ===== VISTA + SP DE PAGINACION ===== */
/* Vista: una fila por factura, con cliente, vendedor, estado y total calculado. */
GO
CREATE VIEW vw_VistaFacturas AS
SELECT f.Num_fac,
       f.Fec_fac,
       c.Raz_soc_cli AS Cliente,
       v.Nom_ven + ' ' + v.Ape_ven AS Vendedor,
       CASE f.Est_fac WHEN 1 THEN 'Pendiente'
                WHEN 2 THEN 'Cancelada'
                ELSE 'Anulada' END AS Estado,
       SUM(d.Can_ven * d.Pre_ven) AS Total
FROM Tb_Factura f
INNER JOIN Tb_Cliente c ON f.Cod_cli = c.Cod_cli
INNER JOIN Tb_Vendedor v ON f.Cod_ven = v.Cod_ven
INNER JOIN Tb_Detalle_Factura d ON f.Num_fac = d.Num_fac
GROUP BY f.Num_fac, f.Fec_fac, c.Raz_soc_cli, v.Nom_ven, v.Ape_ven, f.Est_fac;
GO

```

```
/* SP: devuelve SOLO la pagina pedida usando OFFSET / FETCH. */
CREATE PROCEDURE usp_ListarFacturas_Paginacion
    @pagina INT,
    @tam     INT
AS
BEGIN
    SET NOCOUNT ON;
    SELECT Num_fac, Fec_fac, Cliente, Vendedor, Estado, Total
    FROM    vw_VistaFacturas
    ORDER BY Num_fac
    OFFSET (@pagina - 1) * @tam ROWS      -- salta las paginas previas
    FETCH NEXT @tam ROWS ONLY;          -- devuelve solo esta pagina
END
GO

PRINT 'VentasLeon_S13 aplicado correctamente (Ubigeo, Proveedor, facturas extra, vista y SP de paginacion).';
GO
```

3. Nuevos modelos: Proveedor y Ubigeo

Dentro de **Modelos**, agregue una clase por archivo. Cada clase mapea una tabla.

Modelos \ Proveedor.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace Web_Sesion13.Modelos;

// Entidad que mapea la tabla Tb_Proveedor (Ejemplo 1: CRUD).
[Table("Tb_Proveedor")]
public class Proveedor
{
    [Key]
    [Column("Cod_prv")]
    public string CodPrv { get; set; } = "";

    [Column("Raz_soc_prv")]
    public string RazSocPrv { get; set; } = "";

    [Column("Ruc_prv")]
    public string? RucPrv { get; set; }

    [Column("Dir_prv")]
    public string? DirPrv { get; set; }

    [Column("Cod_dep")]
    public string? CodDep { get; set; }

    [Column("Cod_prov")]
    public string? CodProv { get; set; }

    [Column("Cod_dist")]
    public string? CodDist { get; set; }

    [Column("Est_prv")]
    public int EstPrv { get; set; } // 1 activo, 0 inactivo
}
```

Modelos \ Departamento.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace Web_Sesion13.Modelos;

[Table("Tb_Departamento")]
public class Departamento
{
    [Key]
    [Column("Cod_dep")]
    public string CodDep { get; set; } = "";

    [Column("Nom_dep")]
    public string NomDep { get; set; } = "";
}
```

3. Nuevos modelos (continuación)

Modelos \ Provincia.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace Web_Sesion13.Modelos;

[Table("Tb_Provincia")]
public class Provincia
{
    [Key]
    [Column("Cod_prov")]
    public string CodProv { get; set; } = "";

    [Column("Cod_dep")]
    public string CodDep { get; set; } = "";

    [Column("Nom_prov")]
    public string NomProv { get; set; } = "";
}
```

Modelos \ Distrito.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace Web_Sesion13.Modelos;

[Table("Tb_Distrito")]
public class Distrito
{
    [Key]
    [Column("Cod_dist")]
    public string CodDist { get; set; } = "";

    [Column("Cod_prov")]
    public string CodProv { get; set; } = "";

    [Column("Nom_dist")]
    public string NomDist { get; set; } = "";
}
```

Modelos \ FacturacionAnual.cs

```
namespace Web_Sesion13.Modelos;

// DTO para el grafico (Ejemplo 2): total facturado por anio.
// No es una tabla ni un DbSet; es solo el resultado de una consulta LINQ agrupada.
public class FacturacionAnual
{
    public int Anio { get; set; }
    public decimal Total { get; set; }
}
```

Modelos \ FacturaPagina.cs

```
namespace Web_Sesion13.Modelos;

// Entidad SIN clave (keyless): recibe las filas de la vista vw_VistaFacturas
// que devuelve el SP de paginacion usp_ListarFacturas_Paginacion (Ejemplo 3).
public class FacturaPagina
{
    public string Num_fac { get; set; } = "";
    public DateTime Fec_fac { get; set; }
    public string Cliente { get; set; } = "";
    public string Vendedor { get; set; } = "";
}
```



```
public string Estado { get; set; } = "";  
public decimal Total { get; set; }  
}
```

FacturacionAnual y **FacturaPagina** no son tablas: son clases sencillas que reciben el resultado de una consulta agrupada (Ejemplo 2) y del procedimiento de paginación (Ejemplo 3).

4. Actualizar el DbContext

Reemplace el contenido del contexto para incluir los nuevos `DbSet` y las llaves foráneas. La entidad sin clave del SP se declara con `HasNoKey()`.

Modelos \ VentasLeonContext.cs

```
using Microsoft.EntityFrameworkCore;

namespace Web_Sesion13.Modelos;

// El DbContext es la sesion con la base de datos. Cada DbSet es una tabla
// consultable. Aqui se deja escrito a mano para fines didacticos.
public class VentasLeonContext : DbContext
{
    public VentasLeonContext(DbContextOptions<VentasLeonContext> options)
        : base(options) { }

    // Tablas de la Sesion 12
    public DbSet<Cliente> Clientes { get; set; }
    public DbSet<Vendedor> Vendedores { get; set; }
    public DbSet<Factura> Facturas { get; set; }
    public DbSet<DetalleFactura> DetalleFacturas { get; set; }
    public DbSet<FacturaResumen> FacturasResumen { get; set; }

    // Tablas nuevas de la Sesion 13
    public DbSet<Proveedor> Proveedores { get; set; }
    public DbSet<Departamento> Departamentos { get; set; }
    public DbSet<Provincia> Provincias { get; set; }
    public DbSet<Distrito> Distritos { get; set; }

    // Conjunto sin clave que recibe el resultado del SP de paginacion
    public DbSet<FacturaPagina> FacturasPagina { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Clave compuesta del detalle (Num_fac + Cod_pro)
        modelBuilder.Entity<DetalleFactura>()
            .HasKey(d => new { d.NumFac, d.CodPro });

        // Entidades sin clave: solo reciben filas de un SP o consulta
        modelBuilder.Entity<FacturaResumen>().HasNoKey();
        modelBuilder.Entity<FacturaPagina>().HasNoKey();

        // Relaciones: se indica que las columnas existentes son las llaves foraneas,
        // para que EF Core no invente nombres de columna que no estan en la BD.
        modelBuilder.Entity<Factura>()
            .HasOne(f => f.Cliente).WithMany(c => c.Facturas).HasForeignKey(f => f.CodCli);

        modelBuilder.Entity<Factura>()
            .HasOne(f => f.Vendedor).WithMany(v => v.Facturas).HasForeignKey(f => f.CodVen);

        modelBuilder.Entity<DetalleFactura>()
            .HasOne(d => d.Factura).WithMany(f => f.Detalles).HasForeignKey(d => d.NumFac);
    }
}
```

5. Ampliar la capa de datos (repositorio)

Agregue al repositorio los métodos de los tres ejemplos nuevos. Las páginas nunca tocan la base directamente: siempre llaman a estos métodos.

Datos \ VentasRepositorio.cs

```
using Microsoft.Data.SqlClient;
using Microsoft.EntityFrameworkCore;
using System.Data;
using Web_Sesion13.Modelos;

namespace Web_Sesion13.Datos;

// Capa de datos. Centraliza el acceso a la base de datos mediante EF Core.
// Las paginas NUNCA consultan la BD directamente: siempre llaman a estos metodos.
public class VentasRepositorio
{
    private readonly VentasLeonContext _ctx;

    public VentasRepositorio(VentasLeonContext ctx)
    {
        _ctx = ctx;
    }

    /* ===== SESION 12: CONSULTAS CON LINQ ===== */

    // Suma los importes de las facturas pendientes (estado 1) de un cliente.
    public async Task<decimal> CalcularDeudaClienteLinqAsync(string codigo)
    {
        decimal deuda = await (from f in _ctx.Facturas
                                join d in _ctx.DetalleFacturas on f.NumFac equals d.NumFac
                                where f.CodCli == codigo && f.EstFac == 1
                                select (decimal)(d.CanVen * d.PreVen)).SumAsync();

        return deuda;
    }

    // Lista las facturas de un cliente entre dos fechas (con su detalle).
    public async Task<List<Factura>> ListarFacturasLinqAsync(string codigo, DateTime ini, DateTime fin)
    {
        return await _ctx.Facturas
            .Include(f => f.Detalles)
            .Where(f => f.CodCli == codigo && f.FecFac >= ini && f.FecFac <= fin)
            .OrderBy(f => f.FecFac)
            .ToListAsync();
    }

    /* ===== SESION 12: PROCEDIMIENTOS ALMACENADOS ===== */

    public async Task<List<FacturaResumen>> ListarFacturasSpAsync(string codigo, DateTime ini, DateTime fin)
    {
        return await _ctx.FacturasResumen
            .FromSql($"EXEC usp_ListarFacturasClienteFechas {codigo}, {ini}, {fin}")
            .ToListAsync();
    }

    public async Task<decimal> CalcularDeudaClienteSpAsync(string codigo)
    {
        var pCodigo = new SqlParameter("@codigo", codigo);
        var pDeuda = new SqlParameter("@vdeuda", SqlDbType.Decimal)
        {
            Precision = 12,
            Scale = 2,
            Direction = ParameterDirection.Output
        };

        await _ctx.Database.ExecuteSqlRawAsync(
            "EXEC usp_DeudaCliente @codigo, @vdeuda OUTPUT", pCodigo, pDeuda);

        return pDeuda.Value == DBNull.Value ? 0 : (decimal)pDeuda.Value;
    }

    public async Task<Cliente?> BuscarClienteAsync(string codigo)
```

```

{
    return await _ctx.Clientes.FirstOrDefaultAsync(c => c.CodCli == codigo);
}

/* ===== SESION 13 · EJEMPLO 1: PROVEEDORES (CRUD) ===== */

// Lista proveedores; si el filtro viene vacío trae todos, si no filtra por razón social.
public async Task<List<Proveedor>> ListarProveedoresAsync(string filtro)
{
    filtro ??= "";
    return await _ctx.Proveedores
        .Where(p => filtro == "" || p.RazSocPrv.Contains(filtro))
        .OrderBy(p => p.RazSocPrv)
        .ToListAsync();
}

public async Task<Proveedor?> BuscarProveedorAsync(string codigo)
{
    return await _ctx.Proveedores.FirstOrDefaultAsync(p => p.CodPrv == codigo);
}

public async Task InsertarProveedorAsync(Proveedor prov)
{
    _ctx.Proveedores.Add(prov);
    await _ctx.SaveChangesAsync();
}

public async Task ActualizarProveedorAsync(Proveedor prov)
{
    var p = await _ctx.Proveedores.FindAsync(prov.CodPrv);
    if (p == null) return;

    p.RazSocPrv = prov.RazSocPrv;
    p.RucPrv = prov.RucPrv;
    p.DirPrv = prov.DirPrv;
    p.CodDep = prov.CodDep;
    p.CodProv = prov.CodProv;
    p.CodDist = prov.CodDist;
    p.EstPrv = prov.EstPrv;

    await _ctx.SaveChangesAsync(); // EF Core detecta el cambio y hace el UPDATE
}

// Calcula el próximo código tipo P001, P002... a partir del último registrado.
public async Task<string> SiguienteCodigoProveedorAsync()
{
    var ultimo = await _ctx.Proveedores
        .OrderByDescending(p => p.CodPrv)
        .Select(p => p.CodPrv)
        .FirstOrDefaultAsync();

    int n = 0;
    if (!string.IsNullOrEmpty(ultimo) && ultimo.Length > 1)
        int.TryParse(ultimo.Substring(1), out n);

    return "P" + (n + 1).ToString("000");
}

/* ===== SESION 13 · EJEMPLO 1: UBIGEO (cascada) ===== */

public async Task<List<Departamento>> ListarDepartamentosAsync()
{
    return await _ctx.Departamentos.OrderBy(d => d.NomDep).ToListAsync();
}

public async Task<List<Provincia>> ProvinciasPorDepartamentoAsync(string codDep)
{
    return await _ctx.Provincias
        .Where(p => p.CodDep == codDep)
        .OrderBy(p => p.NomProv)
        .ToListAsync();
}

public async Task<List<Distrito>> DistritosPorProvinciaAsync(string codProv)
{
    return await _ctx.Distritos

```

```

        .Where(d => d.CodProv == codProv)
        .OrderBy(d => d.NomDist)
        .ToListAsync();
    }

    /* ===== SESION 13 · EJEMPLO 2: CONSULTA DE VALOR ===== */

    // Total facturado por anio (no cuenta las anuladas). Alimenta la tabla y el grafico.
    public async Task<List<FacturacionAnual>> FacturacionAnualAsync()
    {
        return await (from f in _ctx.Facturas
            join d in _ctx.DetalleFacturas on f.NumFac equals d.NumFac
            where f.EstFac != 3
            group (decimal)(d.CanVen * d.PreVen) by f.FecFac.Year into g
            orderby g.Key
            select new FacturacionAnual
            {
                Anio = g.Key,
                Total = g.Sum()
            }).ToListAsync();
    }

    /* ===== SESION 13 · EJEMPLO 3: PAGINACION ===== */

    // Total de facturas (para calcular cuantas paginas hay).
    public async Task<int> ContarFacturasAsync()
    {
        return await _ctx.Facturas.CountAsync();
    }

    // Paginacion LOCAL: EF Core traduce Skip/Take a OFFSET/FETCH en SQL Server.
    public async Task<List<Factura>> ListarFacturasLocalAsync(int pagina, int tam)
    {
        return await _ctx.Facturas
            .Include(f => f.Cliente)
            .Include(f => f.Vendedor)
            .Include(f => f.Detalles)
            .OrderBy(f => f.NumFac)
            .Skip((pagina - 1) * tam) // salta las paginas anteriores
            .Take(tam) // toma solo los de esta pagina
            .ToListAsync();
    }

    // Paginacion con SP: la base de datos devuelve solo la pagina pedida.
    public async Task<List<FacturaPagina>> ListarFacturasSpPaginacionAsync(int pagina, int tam)
    {
        return await _ctx.FacturasPagina
            .FromSql($"EXEC usp_ListarFacturas_Paginacion {pagina}, {tam}")
            .ToListAsync();
    }
}

```

6. Diseño: navbar, estilos y página de inicio

Actualice el menú (agrega los tres ejemplos y un desplegable con las páginas de la Sesión 12) y añada al CSS los estilos del gráfico y del paginador.

Pages \ Shared \ _Layout.cshtml

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>@ViewData["Title"] - VentasLeon</title>
  <!-- Bootstrap desde CDN -->
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" />
  <!-- Hoja de estilos propia del sitio -->
  <link rel="stylesheet" href="/css/site.css" asp-append-version="true" />
</head>
<body>
  <!-- Menu superior -->
  <nav class="navbar navbar-expand-lg navbar-dark vl-navbar">
    <div class="container">
      <a class="navbar-brand fw-bold" asp-page="/Index">Ventas&ltspan class="vl-accent">Leon</span></a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#menu">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="menu">
        <ul class="navbar-nav me-auto">
          <li class="nav-item"><a class="nav-link" asp-page="/Index">Inicio</a></li>
          <li class="nav-item"><a class="nav-link" asp-page="/Proveedores">1 · CRUD Proveedores</a></li>
          <li class="nav-item"><a class="nav-link" asp-page="/Grafico">2 · Grafico</a></li>
          <li class="nav-item"><a class="nav-link" asp-page="/Paginacion">3 · Paginacion</a></li>
          <li class="nav-item dropdown">
            <a class="nav-link dropdown-toggle" href="#" role="button" data-bs-toggle="dropdown">Sesion 12</a>
            <ul class="dropdown-menu">
              <li><a class="dropdown-item" asp-page="/Consulta">Consulta LINQ</a></li>
              <li><a class="dropdown-item" asp-page="/Procedimiento">Procedimiento</a></li>
              <li><a class="dropdown-item" asp-page="/Ajax">AJAX</a></li>
            </ul>
          </li>
        </ul>
        <span class="navbar-text vl-stack">EF Core 10 · Bootstrap · Chart.js</span>
      </div>
    </div>
  </nav>

  <main class="container vl-main">
    @RenderBody()
  </main>

  <footer class="vl-footer">
    <div class="container">
      <span>&copy; @DateTime.Now.Year VentasLeon - Desarrollo de Aplicaciones Basico</span>
    </div>
  </footer>

  <!-- jQuery (lo usan Bootstrap y la validacion del cliente) -->
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
  <!-- Script general del sitio -->
  <script src="/js/site-demo.js" asp-append-version="true"></script>
  @await RenderSectionAsync("Scripts", required: false)
</body>
</html>
```

6. Diseño (continuación) · estilos

Agregue al final de `site.css` los estilos nuevos (contenedor del gráfico, paginador y selector de modo). El resto de clases vienen de la Sesión 12.

wwwroot \ css \ site.css (agregar al final)

```
/* Contenedor del grafico (Ejemplo 2) */
.vl-grafico {
  background: #fff;
  border: 1px solid #e3e8ec;
  border-radius: 8px;
  padding: 1rem;
}

/* Paginador (Ejemplo 3) */
.vl-pager {
  display: flex;
  gap: .35rem;
  flex-wrap: wrap;
  align-items: center;
  margin-top: 1rem;
}
.vl-pager a,
.vl-pager span.actual {
  display: inline-block;
  min-width: 38px;
  text-align: center;
  padding: .35rem .6rem;
  border: 1px solid #cfd8dd;
  border-radius: 6px;
  text-decoration: none;
  color: var(--vl-teal);
}
.vl-pager span.actual {
  background: var(--vl-teal);
  border-color: var(--vl-teal);
  color: #fff;
  font-weight: 600;
}
.vl-pager a:hover { background: #eef5f7; }
.vl-pager .deshab {
  color: #aab3b8;
  border-color: #e3e8ec;
  pointer-events: none;
}

/* Selector de modo de paginacion */
.vl-modo { display: inline-flex; gap: .4rem; margin-bottom: 1rem; }
.vl-modo a {
  padding: .4rem .9rem;
  border: 1px solid var(--vl-teal);
  border-radius: 6px;
  text-decoration: none;
  color: var(--vl-teal);
  font-size: .9rem;
}
.vl-modo a.activo { background: var(--vl-teal); color: #fff; }
```

6. Diseño (continuación) · inicio

La página de inicio presenta los tres ejemplos de la sesión.

Pages \ Index.cshtml

```
@page
@model IndexModel
@{
    ViewData["Title"] = "Inicio";
}

<div class="vl-hero">
    <h1>VentasLeon</h1>
    <p class="lead">Sesion 13 · Aplicaciones N capas: CRUD con modal y AJAX, consultas de valor con grafico y paginacion.
</p>
    <p>Tres ejemplos sobre la misma base de datos, ampliando el sitio de la Sesion 12.</p>
</div>

<div class="row g-4">
    <div class="col-md-4">
        <div class="card vl-card">
            <div class="card-body">
                <span class="num">1</span>
                <h5 class="card-title">CRUD de Proveedores</h5>
                <p class="text-muted">Listado en tabla HTML, buscador y modal de Bootstrap para insertar/editar con AJAX,
con combos de ubigeo en cascada.</p>
                <a class="btn btn-vl btn-sm" asp-page="/Proveedores">Abrir ejemplo</a>
            </div>
        </div>
    </div>
    <div class="col-md-4">
        <div class="card vl-card">
            <div class="card-body">
                <span class="num">2</span>
                <h5 class="card-title">Consulta de valor con grafico</h5>
                <p class="text-muted">Facturacion anual mostrada en tabla y en un grafico de barras con Chart.js.</p>
                <a class="btn btn-vl btn-sm" asp-page="/Grafico">Abrir ejemplo</a>
            </div>
        </div>
    </div>
    <div class="col-md-4">
        <div class="card vl-card">
            <div class="card-body">
                <span class="num">3</span>
                <h5 class="card-title">Paginacion</h5>
                <p class="text-muted">Listado de facturas paginado de dos formas: local (Skip/Take) y con procedimiento
almacenado (OFFSET/FETCH).</p>
                <a class="btn btn-vl btn-sm" asp-page="/Paginacion">Abrir ejemplo</a>
            </div>
        </div>
    </div>
</div>

<div class="vl-nota">
    Antes de probar, ejecute una sola vez <strong>VentasLeon.sql</strong> y luego <strong>VentasLeon_S13.sql</strong> en
SQL Server,
    para crear la base, el ubigeo, los proveedores y el procedimiento de paginacion.
</div>
```


6. Diseño (continuación) · inicio (código)

Pages \ Index.cshtml.cs

```
using Microsoft.AspNetCore.Mvc.RazorPages;

namespace Web_Sesion13.Pages;

public class IndexModel : PageModel
{
    public void OnGet() { }
}
```

El script del sitio `site-demo.js` incluye una función `llenarCombo` que usan los combos de ubigeo en cascada del Ejemplo 1.

wwwroot \ js \ site-demo.js

```
// Script general del sitio. Se enlaza una sola vez desde _Layout.cshtml,
// por lo que sus funciones quedan disponibles en cualquier pagina.

// Da formato de moneda en soles a un numero
function formatearSoles(monto) {
    return "S/ " + Number(monto).toFixed(2);
}

// Convierte una cadena a numero, aceptando que venga vacia
function aNumero(texto) {
    var n = parseFloat(texto);
    return isNaN(n) ? 0 : n;
}

// Llena un <select> con una lista de { valor, texto } y un primer item guia
function llenarCombo(combo, lista, guia) {
    combo.innerHTML = "";
    var op = document.createElement("option");
    op.value = "";
    op.textContent = guia || "-- Seleccione --";
    combo.appendChild(op);
    lista.forEach(function (item) {
        var o = document.createElement("option");
        o.value = item.valor;
        o.textContent = item.texto;
        combo.appendChild(o);
    });
}
```

7. Ejemplo 1 · CRUD de Proveedores (vista)

Agregue la Razor Page **Proveedores**. Tiene un buscador, una tabla HTML (los inactivos en rojo), un modal de Bootstrap para insertar/editar y los combos de ubigeo en cascada. Todo se graba por AJAX (fetch), sin recargar.

Pages \ Proveedores.cshtml

```
@page
@model ProveedoresModel
@{
    ViewData["Title"] = "CRUD Proveedores";
}
@Html.AntiForgeryToken()

<div class="panel">
    <p class="tituloForm">Ejemplo 1 · CRUD de Proveedores (tabla HTML + modal + AJAX)</p>

    <div class="row g-3 align-items-end mb-3">
        <div class="col-md-6">
            <label class="form-label">Buscar por razon social</label>
            <input id="filtro" class="form-control" placeholder="Escriba y presione Buscar..." />
        </div>
        <div class="col-md-3">
            <button id="btnBuscar" type="button" class="btn btn-vl w-100">Buscar</button>
        </div>
        <div class="col-md-3">
            <button id="btnNuevo" type="button" class="btn btn-outline-secondary w-100">+ Nuevo proveedor</button>
        </div>
    </div>

    <table class="table tabla-vl">
        <thead>
            <tr>
                <th>Codigo</th><th>Razon social</th><th>RUC</th><th>Estado</th><th class="text-end">Accion</th>
            </tr>
        </thead>
        <tbody id="cuerpoProveedores"></tbody>
    </table>

    <div class="vl-nota">
        La fila de un proveedor inactivo se pinta en rojo (clase <strong>text-danger</strong>). Grabar usa
        <strong>fetch</strong> y no recarga la pagina. La regla de oro sigue: la vista nunca toca la base de datos.
    </div>
</div>

<!-- Modal de Bootstrap para insertar / editar -->
<div class="modal fade" id="modalProveedor" tabindex="-1">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header">
                <h5 class="modal-title" id="tituloModal">Nuevo proveedor</h5>
                <button type="button" class="btn-close" data-bs-dismiss="modal"></button>
            </div>
            <div class="modal-body">
                <input type="hidden" id="codPrv" />
                <div class="mb-2">
                    <label class="form-label">Razon social</label>
                    <input id="razSocPrv" class="form-control" />
                </div>
                <div class="row g-2">
                    <div class="col-md-6 mb-2">
                        <label class="form-label">RUC</label>
                        <input id="rucPrv" class="form-control" maxlength="11" />
                    </div>
                    <div class="col-md-6 mb-2">
                        <label class="form-label">Estado</label>
                        <select id="estPrv" class="form-select">
                            <option value="1">Activo</option>
                            <option value="0">Inactivo</option>
                        </select>
                    </div>
                </div>
            </div>
        </div>
    </div>
</div>
```

```

        <label class="form-label">Direccion</label>
        <input id="dirPrv" class="form-control" />
    </div>
    <hr />
    <p class="text-muted mb-2">Ubigeo (combos en cascada)</p>
    <div class="row g-2">
        <div class="col-md-4 mb-2">
            <label class="form-label">Departamento</label>
            <select id="cboDepartamento" class="form-select"></select>
        </div>
        <div class="col-md-4 mb-2">
            <label class="form-label">Provincia</label>
            <select id="cboProvincia" class="form-select"></select>
        </div>
        <div class="col-md-4 mb-2">
            <label class="form-label">Distrito</label>
            <select id="cboDistrito" class="form-select"></select>
        </div>
    </div>
    <div id="errorModal" class="text-danger mt-2"></div>
</div>
<div class="modal-footer">
    <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">Cancelar</button>
    <button type="button" id="btnGuardar" class="btn btn-vl">Guardar</button>
</div>
</div>
</div>
</div>

@section Scripts {
    <script>
        // instancia del modal de Bootstrap (se abre y cierra desde JavaScript)
        const modal = new bootstrap.Modal(document.getElementById("modalProveedor"));
        const cboDep = document.getElementById("cboDepartamento");
        const cboProv = document.getElementById("cboProvincia");
        const cboDist = document.getElementById("cboDistrito");

        // ---- LISTAR ----
        async function cargarLista() {
            const filtro = document.getElementById("filtro").value.trim();
            const r = await fetch(`?handler=Listar&filtro=${encodeURIComponent(filtro)}`);
            const lista = await r.json();
            const cuerpo = document.getElementById("cuerpoProveedores");
            cuerpo.innerHTML = "";
            lista.forEach(p => {
                const fila = document.createElement("tr");
                // clase CSS condicional: rojo si esta inactivo
                if (p.estPrv === 0) fila.className = "text-danger";
                fila.innerHTML =
                    `<td>${p.codPrv}</td><td>${p.razSocPrv}</td><td>${p.rucPrv || "-"}</td>` +
                    `<td>${p.estPrv === 1 ? "Activo" : "Inactivo"}</td>` +
                    `<td class="text-end"><button class="btn btn-sm btn-outline-secondary"
onclick="editar('${p.codPrv}')">Editar</button></td>`;
                cuerpo.appendChild(fila);
            });
        }

        // ---- UBIGEO EN CASCADA ----
        async function cargarDepartamentos() {
            const r = await fetch(`?handler=Departamentos`);
            llenarCombo(cboDep, await r.json(), "-- Departamento --");
            llenarCombo(cboProv, [], "-- Provincia --");
            llenarCombo(cboDist, [], "-- Distrito --");
        }
        async function cargarProvincias(dep) {
            const r = await fetch(`?handler=Provincias&dep=${encodeURIComponent(dep)}`);
            llenarCombo(cboProv, await r.json(), "-- Provincia --");
            llenarCombo(cboDist, [], "-- Distrito --");
        }
        async function cargarDistritos(prov) {
            const r = await fetch(`?handler=Distritos&prov=${encodeURIComponent(prov)}`);
            llenarCombo(cboDist, await r.json(), "-- Distrito --");
        }
        // al cambiar el combo de departamento se piden sus provincias, sin recargar
        cboDep.addEventListener("change", () => { if (cboDep.value) cargarProvincias(cboDep.value); });
        cboProv.addEventListener("change", () => { if (cboProv.value) cargarDistritos(cboProv.value); });
    </script>
}

```

```

// ---- NUEVO ----
async function nuevo() {
  document.getElementById("tituloModal").textContent = "Nuevo proveedor";
  document.getElementById("errorModal").textContent = "";
  const r = await fetch(`?handler=NuevoCodigo`);
  const data = await r.json();
  document.getElementById("codPrv").value = data.codigo;
  document.getElementById("razSocPrv").value = "";
  document.getElementById("rucPrv").value = "";
  document.getElementById("dirPrv").value = "";
  document.getElementById("estPrv").value = "1";
  await cargarDepartamentos();
  modal.show();
}

// ---- EDITAR ----
async function editar(codigo) {
  document.getElementById("tituloModal").textContent = "Editar proveedor " + codigo;
  document.getElementById("errorModal").textContent = "";
  const r = await fetch(`?handler=Obtener&codigo=${encodeURIComponent(codigo)}`);
  const p = await r.json();
  document.getElementById("codPrv").value = p.codPrv;
  document.getElementById("razSocPrv").value = p.razSocPrv;
  document.getElementById("rucPrv").value = p.rucPrv || "";
  document.getElementById("dirPrv").value = p.dirPrv || "";
  document.getElementById("estPrv").value = p.estPrv;

  // se cargan los combos y se seleccionan los valores guardados, en cascada
  await cargarDepartamentos();
  if (p.codDep) {
    cboDep.value = p.codDep;
    await cargarProvincias(p.codDep);
    if (p.codProv) {
      cboProv.value = p.codProv;
      await cargarDistritos(p.codProv);
      if (p.codDist) cboDist.value = p.codDist;
    }
  }
  modal.show();
}

// ---- GUARDAR (fetch POST con token antiforgery) ----
async function guardar() {
  const dto = {
    codPrv: document.getElementById("codPrv").value,
    razSocPrv: document.getElementById("razSocPrv").value.trim(),
    rucPrv: document.getElementById("rucPrv").value.trim(),
    dirPrv: document.getElementById("dirPrv").value.trim(),
    codDep: cboDep.value,
    codProv: cboProv.value,
    codDist: cboDist.value,
    estPrv: parseInt(document.getElementById("estPrv").value)
  };
  if (dto.razSocPrv === "") {
    document.getElementById("errorModal").textContent = "Ingrese la razon social.";
    return;
  }
  const token = document.querySelector('input[name="__RequestVerificationToken"]').value;
  const r = await fetch(`?handler=Guardar`, {
    method: "POST",
    headers: { "Content-Type": "application/json", "RequestVerificationToken": token },
    body: JSON.stringify(dto)
  });
  const res = await r.json();
  if (res.ok) {
    modal.hide();
    await cargarLista(); // refresca la tabla sin recargar la pagina
  } else {
    document.getElementById("errorModal").textContent = res.msg || "No se pudo guardar.";
  }
}

document.getElementById("btnBuscar").addEventListener("click", cargarLista);
document.getElementById("btnNuevo").addEventListener("click", nuevo);
document.getElementById("btnGuardar").addEventListener("click", guardar);

```

```
    // primera carga  
    cargarLista();  
</script>  
}
```

7. Ejemplo 1 · CRUD de Proveedores (handlers)

El code-behind expone los handlers AJAX (listar, obtener, ubigeo en cascada y guardar). El POST recibe el proveedor como JSON y decide entre insertar o actualizar.

Pages \ Proveedores.cshtml.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Web_Sesion13.Datos;
using Web_Sesion13.Modelos;

namespace Web_Sesion13.Pages;

// Ejemplo 1: CRUD de proveedores. Todos los handlers OnGet... devuelven JSON
// y el guardado se hace por POST con fetch. La vista nunca toca la BD.
public class ProveedoresModel : PageModel
{
    private readonly VentasRepositorio _repo;
    public ProveedoresModel(VentasRepositorio repo) { _repo = repo; }

    public void OnGet() { }

    // ?handler=Listar -> lista de proveedores (opcionalmente filtrada)
    public async Task<IActionResult> OnGetListarAsync(string? filtro)
    {
        var lista = await _repo.ListarProveedoresAsync(filtro ?? "");
        var datos = lista.Select(p => new
        {
            p.CodPrv, p.RazSocPrv, p.RucPrv, p.EstPrv
        });
        return new JsonResult(datos);
    }

    // ?handler=Obtener -> un proveedor para cargar el modal de edicion
    public async Task<IActionResult> OnGetObtenerAsync(string codigo)
    {
        var p = await _repo.BuscarProveedorAsync(codigo);
        if (p == null) return new JsonResult(new { });
        return new JsonResult(new
        {
            p.CodPrv, p.RazSocPrv, p.RucPrv, p.DirPrv,
            p.CodDep, p.CodProv, p.CodDist, p.EstPrv
        });
    }

    // ?handler=NuevoCodigo -> proximo codigo correlativo (P001, P002...)
    public async Task<IActionResult> OnGetNuevoCodigoAsync()
    {
        return new JsonResult(new { codigo = await _repo.SiguienteCodigoProveedorAsync() });
    }

    // ?handler=Departamentos -> combo de departamentos { valor, texto }
    public async Task<IActionResult> OnGetDepartamentosAsync()
    {
        var lista = await _repo.ListarDepartamentosAsync();
        return new JsonResult(lista.Select(d => new { valor = d.CodDep, texto = d.NomDep }));
    }

    // ?handler=Provincias -> provincias del departamento elegido
    public async Task<IActionResult> OnGetProvinciasAsync(string dep)
    {
        var lista = await _repo.ProvinciasPorDepartamentoAsync(dep);
        return new JsonResult(lista.Select(p => new { valor = p.CodProv, texto = p.NomProv }));
    }

    // ?handler=Distritos -> distritos de la provincia elegida
    public async Task<IActionResult> OnGetDistritosAsync(string prov)
    {
        var lista = await _repo.DistritosPorProvinciaAsync(prov);
        return new JsonResult(lista.Select(d => new { valor = d.CodDist, texto = d.NomDist }));
    }
}
```

```
// POST ?handler=Guardar -> inserta o actualiza segun exista el codigo
public async Task<IActionResult> OnPostGuardarAsync([FromBody] Proveedor prov)
{
    if (prov == null || string.IsNullOrEmpty(prov.RazSocPrv))
        return new JsonResult(new { ok = false, msg = "Datos incompletos." });

    var existe = await _repo.BuscarProveedorAsync(prov.CodPrv);
    if (existe == null)
        await _repo.InsertarProveedorAsync(prov);
    else
        await _repo.ActualizarProveedorAsync(prov);

    return new JsonResult(new { ok = true });
}
}
```

El guardado por AJAX usa el token antiforgery: [Program.cs](#) configura el nombre de cabecera y el JavaScript lo envía en cada POST.

Web_Sesion13 \ Program.cs

```
using Microsoft.EntityFrameworkCore;
using Web_Sesion13.Datos;
using Web_Sesion13.Modelos;

// Punto de entrada de la aplicacion. Aqui se registran los servicios
// y se arma la tubería de peticiones (middlewares).
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddRazorPages();

// El AJAX de guardado (POST) envia el token antiforgery en una cabecera.
builder.Services.AddAntiforgery(o => o.HeaderName = "RequestVerificationToken");

// Se registra el DbContext de EF Core con la cadena de conexion de appsettings.json.
// A partir de aqui, cualquier pagina o clase puede recibir el contexto por inyeccion.
builder.Services.AddDbContext<VentasLeonContext>(opciones =>
    opciones.UseSqlServer(builder.Configuration.GetConnectionString("VentasLeon")));

// La capa de datos: expone los metodos de consulta (LINQ y procedimientos almacenados).
builder.Services.AddScoped<VentasRepositorio>();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles(); // permite servir css y js desde wwwroot
app.UseRouting();
app.UseAuthorization();
app.MapRazorPages();

app.Run();
```

8. Ejemplo 2 · Consulta de valor con gráfico

Agregue la Razor Page **Grafico**. Muestra la facturación anual en una tabla y en un gráfico de barras con Chart.js. El total se calcula en el servidor con LINQ.

Pages \ Grafico.cshtml

```
@page
@model GraficoModel
@{
    ViewData["Title"] = "Facturacion anual";
}

<div class="panel">
    <p class="tituloForm">Ejemplo 2 · Consulta de valor con grafico (EF Core + Chart.js)</p>

    <p class="text-muted">
        Total facturado por año (sin contar las anuladas). El cálculo se hace en el
        servidor con LINQ (<strong>GroupBy</strong>) y el resultado se muestra en una tabla y en un
        grafico de barras.
    </p>

    @if (Model.Facturacion.Count > 0)
    {
        <div class="row g-4 mt-1">
            <div class="col-md-5">
                <table class="table tabla-vl">
                    <thead>
                        <tr><th>Año</th><th class="text-end">Total facturado</th></tr>
                    </thead>
                    <tbody>
                        @foreach (var fila in Model.Facturacion)
                        {
                            <tr>
                                <td>@fila.Anio</td>
                                <td class="text-end">$ @fila.Total.ToString("0.00")</td>
                            </tr>
                        }
                    </tbody>
                    <tfoot>
                        <tr class="fw-bold">
                            <td>Total</td>
                            <td class="text-end">$ @Model.Facturacion.Sum(f => f.Total).ToString("0.00")</td>
                        </tr>
                    </tfoot>
                </table>
            </div>
            <div class="col-md-7">
                <div class="vl-grafico">
                    <canvas id="grafico"></canvas>
                </div>
            </div>
        </div>

        <div class="vl-nota mt-3">No hay facturacion registrada.</div>
    }

    <div class="vl-nota">
        El servidor calcula los totales con LINQ y los entrega a la vista. <strong>Chart.js</strong>
        (una libreria de JavaScript) recibe esos datos y dibuja el grafico en el navegador,
        reemplazando al antiguo control <strong>Chart</strong> de Web Forms.
    </div>
</div>

@section Scripts {
    <!-- Chart.js desde CDN -->
    <script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.3/dist/chart.umd.min.js"></script>
    <script>
        // los datos vienen del servidor (calculados con LINQ) y se pasan al grafico
        const anios = @Html.Raw(System.Text.Json.JsonSerializer.Serialize(Model.Anios));
        const totales = @Html.Raw(System.Text.Json.JsonSerializer.Serialize(Model.Totales));
```



```
    if (anios.length > 0) {  
      new Chart(document.getElementById("grafico"), {  
        type: "bar",  
        data: {  
          labels: anios,  
          datasets: [{  
            label: "Total facturado (S/)",  
            data: totales,  
            backgroundColor: "#1E88A5"  
          }]  
        },  
        options: {  
          responsive: true,  
          plugins: { legend: { display: true } },  
          scales: { y: { beginAtZero: true } }  
        }  
      });  
    }  
  }  
</script>  
}
```

8. Ejemplo 2 · Consulta de valor (código)

Pages \ Grafico.cshtml.cs

```
using Microsoft.AspNetCore.Mvc.RazorPages;
using Web_Sesion13.Datos;
using Web_Sesion13.Modelos;

namespace Web_Sesion13.Pages;

// Ejemplo 2: consulta de valor (facturacion anual) mostrada en tabla y en grafico.
public class GraficoModel : PageModel
{
    private readonly VentasRepositorio _repo;

    public GraficoModel(VentasRepositorio repo)
    {
        _repo = repo;
    }

    public List<FacturacionAnual> Facturacion { get; set; } = new();

    // listas planas que se serializan a JSON para Chart.js
    public List<string> Anios { get; set; } = new();
    public List<decimal> Totales { get; set; } = new();

    public async Task OnGetAsync()
    {
        // se pide el calculo a la capa de datos (LINQ con GroupBy)
        Facturacion = await _repo.FacturacionAnualAsync();

        // se arman las listas que necesita el grafico
        Anios = Facturacion.Select(f => f.Anio.ToString()).ToList();
        Totales = Facturacion.Select(f => f.Total).ToList();
    }
}
```

Los datos se pasan del servidor al gráfico serializándolos a JSON con `System.Text.Json`. Chart.js recibe las etiquetas (años) y los valores (totales) y dibuja el gráfico en el navegador.

9. Ejemplo 3 · Paginación (vista)

Agregue la Razor Page **Paginacion**. Muestra las facturas por páginas y permite elegir la forma de paginar: local (Skip/Take) o con procedimiento almacenado (OFFSET/FETCH).

Pages \ Paginacion.cshtml

```
@page
@model PaginacionModel
@{
    ViewData["Title"] = "Paginacion";
}

<div class="panel">
    <p class="tituloForm">Ejemplo 3 · Paginacion (local con Skip/Take y con procedimiento almacenado)</p>

    <p class="text-muted">
        El listado de facturas se muestra por p ginas de @Model.Tam en @Model.Tam. Elija la forma
        de paginar: en memoria con LINQ (<strong>Skip/Take</strong>) o dejando que la base de datos
        devuelva solo la p gina con un procedimiento almacenado (<strong>OFFSET/FETCH</strong>).
    </p>

    <!-- selector de modo: mantiene la pagina en 1 al cambiar -->
    <div class="vl-modo">
        <a class="@((Model.Modo == "local" ? "activo" : ""))"
            asp-page="/Paginacion" asp-route-modo="local" asp-route-pagina="1">Local (Skip/Take)</a>
        <a class="@((Model.Modo == "sp" ? "activo" : ""))"
            asp-page="/Paginacion" asp-route-modo="sp" asp-route-pagina="1">Procedimiento (OFFSET/FETCH)</a>
    </div>

    <table class="table tabla-vl">
        <thead>
            <tr>
                <th>N deg; Factura</th>
                <th>Fecha</th>
                <th>Cliente</th>
                <th>Vendedor</th>
                <th>Estado</th>
                <th class="text-end">Total</th>
            </tr>
        </thead>
        <tbody>
            @if (Model.Modo == "sp")
            {
                @foreach (var f in Model.FacturasSp)
                {
                    <tr>
                        <td>@f.Num_fac</td>
                        <td>@f.Fec_fac.ToString("dd/MM/yyyy")</td>
                        <td>@f.Cliente</td>
                        <td>@f.Vendedor</td>
                        <td>@f.Estado</td>
                        <td class="text-end">S/ @f.Total.ToString("0.00")</td>
                    </tr>
                }
            }
            else
            {
                @foreach (var f in Model.FacturasLocal)
                {
                    <tr>
                        <td>@f.NumFac</td>
                        <td>@f.FecFac.ToString("dd/MM/yyyy")</td>
                        <td>@(f.Cliente?.RazSocCli)</td>
                        <td>@(f.Vendedor?.NomVen) @(f.Vendedor?.ApeVen)</td>
                        <td>@f.EstadoTexto</td>
                        <td class="text-end">S/ @(f.Detalles.Sum(d => d.CanVen * d.PreVen).ToString("0.00"))</td>
                    </tr>
                }
            }
        </tbody>
    </table>
```

```

<!-- paginador: cada enlace conserva el modo actual y cambia la pagina -->
<nav class="vl-pager">
    @if (Model.Pagina > 1)
    {
        <a asp-page="/Paginacion" asp-route-modo="@Model.Modo" asp-route-pagina="@((Model.Pagina - 1))">&laquo;</a>
    }
    else
    {
        <span class="actual deshab">&laquo;</span>
    }

    @for (int i = 1; i <= Model.TotalPaginas; i++)
    {
        if (i == Model.Pagina)
        {
            <span class="actual">@i</span>
        }
        else
        {
            <a asp-page="/Paginacion" asp-route-modo="@Model.Modo" asp-route-pagina="@i">@i</a>
        }
    }

    @if (Model.Pagina < Model.TotalPaginas)
    {
        <a asp-page="/Paginacion" asp-route-modo="@Model.Modo" asp-route-pagina="@((Model.Pagina + 1))">&raquo;</a>
    }
    else
    {
        <span class="actual deshab">&raquo;</span>
    }
</nav>

<p class="text-muted mt-2">
    P&aaacute;gina @Model.Pagina de @Model.TotalPaginas &middot; @Model.Total facturas en total.
</p>

<div class="vl-nota">
    Las dos formas dan el mismo resultado en pantalla. La diferencia est&aaacute; en d&oacute;nde se
    recorta: en <strong>local</strong> EF Core traduce Skip/Take a OFFSET/FETCH; con el
    <strong>procedimiento</strong>, es la base de datos la que devuelve solo las filas de la
    p&aaacute;gina pedida. Para vol&uacute;menes grandes, se recomienda el procedimiento.
</div>
</div>

```

9. Ejemplo 3 · Paginación (código)

Pages \ Paginacion.cshtml.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Web_Sesion13.Datos;
using Web_Sesion13.Modelos;

namespace Web_Sesion13.Pages;

// Ejemplo 3: paginacion del listado de facturas, en dos modos (local y con SP).
public class PaginacionModel : PageModel
{
    private readonly VentasRepositorio _repo;

    public PaginacionModel(VentasRepositorio repo)
    {
        _repo = repo;
    }

    // El modo y la pagina llegan por la URL (?modo=sp&pagina=2)
    [BindProperty(SupportsGet = true)]
    public string Modo { get; set; } = "local";

    [BindProperty(SupportsGet = true)]
    public int Pagina { get; set; } = 1;

    public int Tam { get; } = 5; // tamaño de pagina
    public int Total { get; set; } // total de facturas
    public int TotalPaginas { get; set; } // cuantas paginas hay

    public List<Factura> FacturasLocal { get; set; } = new();
    public List<FacturaPagina> FacturasSp { get; set; } = new();

    public async Task OnGetAsync()
    {
        // se cuenta el total para saber cuantas paginas hay
        Total = await _repo.ContarFacturasAsync();
        TotalPaginas = (int)Math.Ceiling(Total / (double)Tam);
        if (TotalPaginas == 0) TotalPaginas = 1;

        // se corrige la pagina si viene fuera de rango
        if (Pagina < 1) Pagina = 1;
        if (Pagina > TotalPaginas) Pagina = TotalPaginas;

        // segun el modo elegido se llama a la capa de datos correspondiente
        if (Modo == "sp")
            FacturasSp = await _repo.ListarFacturasSpPaginacionAsync(Pagina, Tam);
        else
            FacturasLocal = await _repo.ListarFacturasLocalAsync(Pagina, Tam);
    }
}
```

Las dos formas dan el mismo resultado en pantalla. En **local** EF Core traduce Skip/Take a OFFSET/FETCH; con el **procedimiento** es la base de datos la que devuelve solo la página pedida.

10. Probar la aplicación

Paso 1 Verifique que ejecutó `VentasLeon_S13.sql` sobre la base VentasLeon y que la cadena de `appsettings.json` apunta a su servidor (use `Server=. \SQLEXPRESS` si esa es su instancia).

Paso 2 Presione F5. Se abre la página de inicio con los tres ejemplos.

Paso 3 1 · CRUD Proveedores: pruebe el buscador, cree uno nuevo con "Nuevo" y edite uno existente. Al elegir Departamento se cargan sus Provincias y luego los Distritos, sin recargar. Los inactivos salen en rojo.

Paso 4 2 · Gráfico: verá la facturación por año en tabla y en un gráfico de barras.

Paso 5 3 · Paginación: recorra las páginas y cambie entre "Local" y "Procedimiento". El resultado es el mismo; cambia dónde se recorta.

Con esto quedan cubiertos los tres temas de la sesión: un CRUD web moderno (tabla HTML + modal de Bootstrap + AJAX con combos en cascada), las consultas de valor con gráficos, y la paginación local y con procedimiento almacenado.